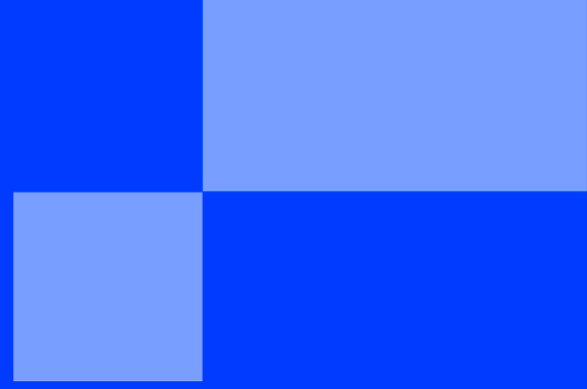




WORKSHOP C

< PREMIERS PAS EN C />

WORKSHOP C

Préliminaires

Langage : C

La totalité de vos fichiers sources, à l'exception des fichiers inutiles (binaires, fichiers objets, temporaires, etc.), doit être présente dans votre dépôt.

✓ Compilez tous vos fichiers avec :

```
gcc *.c
```

- ✓ Faites attention aux warnings.
- ✓ Lisez attentivement les messages d'erreur.
- ✓ N'hésitez pas à tester de petites choses.
- ✓ Posez des questions.
- ✓ Cassez des trucs.
- ✓ Essayez de comprendre ce que fait réellement la machine.



Ce workshop est un workshop de découverte.
Le but n'est PAS de maîtriser le C en 1h30.

Le but est de comprendre :

- ce qu'est la compilation;
- pourquoi le C est différent de Python/JavaScript;
- ce qu'est réellement la mémoire;
- comment fonctionne un langage bas niveau.

Rappel compilation

```
gcc main.c -o main  
./main
```

Warnings fréquents

'=' vs '=='

```
if (a == b)
```

PAS :

```
if (a = b)
```

Oubli du ;

```
int age = 18;
```

Différence entre :

```
'A'
```

et :

```
"A"
```

Les tableaux ont une taille fixe

```
int numbers[5];
```

Une string en C est :

- ✓ un tableau de caractères ;
 - ✓ terminé par “.
-

Task 00 — Hello World



Delivery : hello_world.c

Créer un programme affichant :

```
Hello World
```

Puis le modifier pour afficher :

```
Hello <votre_prénom>
```

Concepts :

- `main`
 - `printf`
 - compilation
 - `return 0`
-

Task 01 — Variables



Delivery : variables.c

Créer des variables stockant :

- votre âge ;
- votre taille ;
- la première lettre de votre prénom.

Afficher les variables avec `printf`.

Concepts :

- `int`
 - `float`
 - `char`
 - spécificateurs de format
-

Task 02 — Conditions



Delivery : conditions.c

Créer un programme qui :

- stocke un nombre ;
- affiche si le nombre est positif, négatif ou nul.

Concepts :

- `if`
 - `else`
 - opérateurs de comparaison
-

Task 03 — Boucles



Delivery : loops.c

Afficher :

- les nombres de 1 à 20 ;
- uniquement les nombres pairs ;
- la somme des nombres de 1 à 10.

Concepts :

- `for`
 - incrémentation
 - accumulation
-

Task 04 — Fonctions



Delivery : functions.c

Créer :

- une fonction additionnant deux nombres ;
- une fonction multipliant deux nombres ;
- une fonction retournant la plus grande valeur.

Concepts :

- paramètres
 - valeur de retour
 - prototypes
-

Task 05 — Tableaux



Delivery : arrays.c

Créer un tableau de 5 entiers.

Afficher :

- toutes les valeurs ;
- la somme du tableau ;
- la plus grande valeur.

Concepts :

- tableaux
 - indexation
 - boucles
-

Task 06 — Strings



Delivery : strings.c

Créer une string contenant votre prénom.

Afficher :

- la string complète ;
- chaque caractère sur une ligne différente.

Concepts :

- `char *`
 - strings
 - “
-

Task 07 — Pointeurs



Delivery : pointers.c

Créer :

- une variable entière ;
- un pointeur pointant vers cette variable.

Afficher :

- la valeur ;
- l'adresse ;
- la valeur via le pointeur.

Modifier la valeur à travers le pointeur.

Concepts :

- &
- *
- adresses mémoire
- déréférencement

Task 08 — Swap



Delivery : swap.c

Créer une fonction échangeant les valeurs de deux entiers avec des pointeurs.

Comportement attendu :

```
a = 10
b = 42

swap(a, b)

a = 42
b = 10
```


Concepts :

- pointeurs
 - paramètres de fonction
 - manipulation mémoire
-

Task 09 — Allocation dynamique



Delivery : malloc.c

Créer un programme qui :

- demande une taille à l'utilisateur ;
- alloue dynamiquement un tableau d'entiers ;
- remplit le tableau ;
- affiche les valeurs ;
- affiche la somme ;
- libère la mémoire.

Fonctions autorisées :

- `malloc`
- `free`
- `printf`
- `scanf`

Concepts :

- heap
 - allocation dynamique
 - pointeurs
 - `sizeof`
 - memory leaks
-

Rappel important

```
int *array = malloc(sizeof(int) * size);
```

`malloc` retourne une adresse mémoire.

Toujours libérer la mémoire allouée.

```
free(array);
```

Comportement dangereux

```
free(array);  
array[0] = 42;
```

Bonus

Si vous avez terminé en avance :

- créer une mini calculatrice ;
- créer un jeu de devinette ;
- afficher une pyramide ASCII ;
- compter les voyelles d'une string.

Note finale

Le C est difficile parce qu'il montre des choses normalement cachées.

Comprendre le C aide énormément à comprendre :

- Python ;
 - JavaScript ;
 - les performances ;
 - la mémoire ;
 - les systèmes ;
 - le fonctionnement réel des programmes.
-

v1

{EPITECH}